Below is a **team-friendly setup guide** for deploying SmartOps via the **umbrella Helm chart** at `platform/helm/smartops`.

SmartOps Kubernetes Deployment Guide (Helm Umbrella)

0) What this deploys (from Chart.yaml you verified)

Umbrella chart: `smartops` (v0.1.0) with dependencies:

- `prometheus` (kube-prometheus-stack) *(optional via values)*
- `loki` (optional)
- `otelcol` (optional)
- `tempo` (optional)
- `ingress-metrics` (local subchart)
- `erp-simulator` (local subchart)
- `orchestrator` (local subchart)

✅ Verified from `platform/helm/smartops/Chart.yaml`.

Note: `agent-detect`, `agent-diagnose`, `policy_engine`, `dashboard` are in `apps/` and also have raw YAML in `k8s/` — but your umbrella chart currently packages **erp-simulator + orchestrator + telemetry**.
So this guide focuses on the **umbrella chart** (the correct "platform deployment path").

---

1) Prerequisites (cluster + tools)

Required tools

```
kubectl version --client
helm version
docker version
```

Required cluster

- Any Kubernetes cluster works.
- For Docker Desktop Kubernetes: make sure context is correct:

```
kubectl config current-context
kubectl cluster-info
```

---

2) Pick a namespace (recommended: smartops-dev)

Your `values.yaml` has namespace commented (`# namespace: default`).
So **you must set namespace in values**, or Helm will use the release namespace behavior.

✅ Recommended: use `smartops-dev`.

```
kubectl create namespace smartops-dev --dry-run=client -o yaml | kubectl
apply -f -
```

---

3) Build Helm dependencies (very important)

Because your Chart.yaml uses `version: "*"` for remote charts, Helm will pull the latest matching versions unless Chart.lock is present.

From repo root:

```
cd platform/helm/smartops
helm dependency build
```

If you ever see dependency errors:

```
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo add open-telemetry https://open-
telemetry.github.io/opentelemetry-helm-charts
helm repo update
helm dependency build
```

---

4) Configure images (critical)

You confirmed images are templated in subcharts (`orchestrator`, `erp-simulator`, `odoo`), and `out.yaml` shows examples like:

- `ghcr.io/vgamaka/smartops-orchestrator:dev-otel2`
- `ghcr.io/vgamaka/erp-simulator:dev-otel1`
- `odoo:17.0`, `postgres:15` etc.

So the deployment strategy depends on your team:

Option A (easiest): Use GHCR / remote images

Set image repo + tag in `platform/helm/smartops/values.dev.yaml` (recommended for teammates).

Option B (local Docker Desktop images)

If teammates build locally and don't push:

- set `imagePullPolicy: IfNotPresent` in chart values
- use repository names that exist locally in Docker Desktop

---

5) Install SmartOps umbrella chart

From repo root:

```
helm upgrade --install smartops platform/helm/smartops \
  -n smartops-dev \
  -f platform/helm/smartops/values.dev.yaml
```

Verify:

```
kubectl -n smartops-dev get pods
kubectl -n smartops-dev get svc
kubectl -n smartops-dev get deploy
```

---

6) Verify Orchestrator RBAC (you already have correct template)

Your `orchestrator-rbac.yaml` grants:

- pods/log/events get/list/watch/delete
- deployments/replicasets get/list/watch/patch/update
- deployments/scale patch/update
- namespaces get/list

So once deployed, confirm the ServiceAccount exists:

```
kubectl -n smartops-dev get sa | grep orchestrator
kubectl -n smartops-dev get role,rolebinding | grep orchestrator
```

---

7) Health checks (the fastest “is it alive?” tests)

Orchestrator

Find service name:

```
kubectl -n smartops-dev get svc | grep orchestrator
```

Port-forward it (example, adjust port if your svc differs):

```
kubectl -n smartops-dev port-forward svc/smartops-orchestrator 8001:8001
```

Then:

```
curl http://localhost:8001/healthz
```

ERP Simulator

```
kubectl -n smartops-dev get svc | grep erp
kubectl -n smartops-dev port-forward svc/smartops-erp-simulator 8010:8010
curl http://localhost:8010/healthz
```

---

8) Telemetry stack verification (if enabled in values)

If prometheus/grafana enabled (kube-prometheus-stack), check:

```
kubectl -n smartops-dev get pods | egrep -i
"prometheus|grafana|alertmanager"
```

If you exposed Grafana under a subpath before (like `/grafana`) using Ingress, verify ingress:

```
kubectl -n smartops-dev get ingress
```

---

9) Deploy “apps/” components (team modules)

Your umbrella chart doesn’t currently include these apps:

- `apps/agent-detect`
- `apps/agent-diagnose`
- `apps/policy_engine`
- `apps/dashboard`

So teammates can deploy them using:

- **their own Helm subcharts** (recommended future), OR
- **raw YAML** from `k8s/` (quick MVP), OR
- add them as subcharts under `platform/helm/smartops/charts/`

Current quick MVP path (raw YAML)

```
kubectl -n smartops-dev apply -f k8s/policy-engine-deploy.yaml
kubectl -n smartops-dev apply -f k8s/policy-engine-svc.yaml
kubectl -n smartops-dev apply -f k8s/agent-detect.yaml
kubectl -n smartops-dev apply -f k8s/agent-diagnose.yaml
```

Then verify:

```
kubectl -n smartops-dev get pods
kubectl -n smartops-dev logs deploy/smartops-policy-engine --tail=50
```

---

10) Known “gotchas” you already hit (so teammates don’t waste time)

✅ zsh `?` must be quoted

This fails:

```
curl http://localhost:8002/v1/policy/audit/latest?n=5
```

Use:

```
curl "http://localhost:8002/v1/policy/audit/latest?n=5"
```

✅ Policy-engine port confusion

You verified container port is:

```
kubectl -n smartops-dev get deploy smartops-policy-engine -o
jsonpath='{.spec.template.spec.containers[0].ports}'
# shows 8002
```

So for local testing:

```
curl "http://localhost:8002/healthz"
```

✅ Policy engine doesn’t include lifecycle/simulate endpoints yet

OpenAPI shows only:

- `/v1/policy/evaluate`
- `/v1/policy/audit/latest`

So if anyone expects “reload/simulate/versioning endpoints” — not present yet.

What I need next to finalize a “ready-to-share” guide for your team

To make this guide **100% copy-paste deployable** for teammates, run these 4 commands and paste outputs:

1) Show key `values.dev.yaml` toggles (telemetry + namespace + enable flags)

```
sed -n '1,260p' platform/helm/smartops/values.dev.yaml
```

2) Confirm release resource names (best from helm template)

```
helm template smartops platform/helm/smartops -n smartops-dev -f
platform/helm/smartops/values.dev.yaml | head -n 60
```

3) Show orchestrator + erp-simulator image values files (to document exact overrides)

```
sed -n '1,120p' platform/helm/smartops/charts/orchestrator/values.yaml
sed -n '1,120p' platform/helm/smartops/charts/erp-simulator/values.yaml
```

4) If you want Dashboard included in the guide (it has its own YAML)

Paste:

```
sed -n '1,200p' apps/dashboard/deployment.yaml
sed -n '1,200p' apps/dashboard/service.yaml
sed -n '1,200p' apps/dashboard/ingress.yaml
```

Once I have those, I'll output a **final team guide** with:

- exact `helm upgrade --install ...` command
- exact `values.dev.yaml` edits to make
- exact expected pods/services list
- health checks + port-forward shortcuts
- troubleshooting section (including your real issues: zsh `?`, policy port 8002, missing curl in containers, etc.)